

Image Captioning with Gen AI

Project Documentation • Python • Hugging Face Transformers • BLIP • Gradio

1. Project Overview

This project is an AI-powered image captioning application that accepts an image from the user and automatically generates a short textual description of what the image contains. The application uses the pre-trained BLIP (Bootstrapping Language-Image Pre-training) model through the Hugging Face Transformers library and provides a simple web interface using Gradio.

2. What We Built

The application follows a simple image-to-text workflow:

- **Upload an image:** The user selects an image through the Gradio interface.
- **Image preprocessing:** BLIPProcessor prepares the image in the format required by the model.
- **AI caption generation:** BlipForConditionalGeneration generates text describing the image.
- **Text decoding:** The generated token IDs are converted into a readable caption.
- **Display the result:** The caption is returned to the Gradio interface as text.

3. Technologies and Libraries Used

Technology / Library	Purpose in the Project
Python	Main programming language used to build the application logic.
Gradio	Creates the interactive user interface for image upload and text output.
Hugging Face Transformers	Provides the BLIP processor and image-captioning model.
BLIP	Pre-trained multimodal model used to generate image descriptions.
PIL Image	Used through Gradio's PIL image type so the uploaded image can be processed.
PyTorch tensors	The processor returns model-ready tensors using <code>return_tensors='pt'</code> .

4. Important Code Components

BlipProcessor — Processes the uploaded image and prepares the input tensors required by the BLIP model.

BlipForConditionalGeneration — Loads the pre-trained BLIP image-captioning model used to generate the caption.

generate_caption(image) — Receives an image, processes it, generates model output, and decodes the output into readable text.

caption_image(image) — Acts as the interface function and uses try/except to return a user-friendly error message if caption generation fails.

gr.Interface(...) — Creates the Gradio application with an image input, text output, title, and description.

`iface.launch(server_name='127.0.0.1')` — Starts the application locally on the user's computer.

5. How the AI Part Works

The project does not train a new model from scratch. Instead, it uses the pre-trained **Salesforce/blip-image-captioning-base** model. The model has learned associations between visual information and language and can generate a textual caption from an image.

The key model flow in the code is:

Image → **BlipProcessor** → **Model.generate()** → **Processor.decode()** → **Caption**

6. User Interface

Gradio is used to avoid building a separate frontend from scratch. The interface contains an image upload component and a text output component. The user uploads an image, the application sends it through the caption-generation function, and the generated caption is displayed on the page.

7. Error Handling

The application uses a try/except block inside **caption_image()**. If an exception occurs while processing the image or generating the caption, the application returns an error message instead of terminating the interface function unexpectedly.

8. Benefits and Practical Uses

- **Accessibility:** Automatically generated image descriptions can support applications where textual descriptions of images are useful.
- **Content workflows:** Captions can help create initial descriptions for uploaded visual content.
- **Learning:** The project demonstrates a practical use of a pre-trained Generative AI / multimodal model.
- **Rapid prototyping:** Hugging Face provides access to a ready-to-use model, while Gradio makes it possible to build an interactive demo quickly.
- **Reduced development effort:** A pre-trained model can be used instead of building and training an image-captioning model from the beginning.

9. Skills Demonstrated

- Python programming
- Generative AI concepts
- Computer vision and image-to-text generation
- Hugging Face Transformers
- BLIP model integration
- Model input preprocessing and output decoding
- Gradio interface development
- Exception handling
- Working with pre-trained AI models

10. Project Workflow Summary

Step	Action
1	User uploads an image.
2	Gradio passes the image to the captioning function.
3	BlipProcessor preprocesses the image.
4	BLIP generates caption tokens using <code>model.generate()</code> .
5	The processor decodes the output tokens into text.
6	The generated caption is displayed to the user.

11. Project Summary

Image Captioning with Gen AI is a practical demonstration of integrating a pre-trained multimodal AI model into a Python application. It combines Hugging Face Transformers for model integration, BLIP for image-to-text generation, and Gradio for an accessible interactive interface. The project shows how existing Generative AI models can be integrated into useful applications without training a model from scratch.

Note: This documentation describes the functionality supported by the provided code. It does not claim a measured accuracy percentage, production deployment, or user-scale performance that is not demonstrated by the code itself.